



API Security Risk Analysis Report

Task 3 — Read-only security assessment of a public REST API:
authentication testing, data exposure analysis, authorization checks, and
remediation guidance.



PREPARED BY	Yousef Ahmed Fawzy
CIN ID	FIT/JUN26/CS9482
INTERNSHIP	Cyber Security Internship (Remote)
PERIOD	29 June 2026 – 29 July 2026
REPOSITORY	github.com/youseffawzyy27/ FUTURE_CS_03
REPORT DATE	July 2026



Table of Contents

1	Executive Summary
2	Methodology & Tools Used
3	Finding 1 — Open/Unauthenticated Endpoints
4	Finding 2 — Excessive Data Exposure
5	Authentication Flow Testing
6	Finding 3 — Broken Authorization (IDOR)
7	Finding 4 — Missing Rate Limiting
8	Input Validation Testing
9	Risk Summary & Remediation Plan
10	Conclusion & References

Scope note: This assessment used DummyJSON (dummyjson.com), a free public test API, in place of the originally suggested ReqRes, since ReqRes now requires a paid account and API key for all endpoints. All testing was read-only (GET/POST) with no exploitation, flooding, or attacks against private systems.

Executive Summary

APIs power the majority of modern applications — mobile apps, SaaS dashboards, and service integrations all rely on them. This report presents a read-only security risk analysis of a public test API, performed in the style of a professional AppSec consultant engagement. The goal was to identify common API security weaknesses, assess their real-world business impact, and provide clear remediation guidance.

- One API was tested end-to-end across all 6 risk categories defined in the assessment scope.
- Two critical/high-severity findings were identified: severe excessive data exposure and a broken authorization (IDOR) vulnerability.
- One medium-severity finding: absence of rate limiting, which compounds the impact of the other issues.
- The API's authentication layer itself (login, token validation, input handling) was found to be properly implemented.

Methodology & Tools Used

Analysis Workflow

- **Select API** — chose a public, no-signup test API suitable for security education
- **Review Documentation** — studied available endpoints, authentication model, and resources
- **Test Endpoints** — sent real GET/POST requests using Postman
- **Inspect** — examined authentication requirements, response headers, and returned data
- **Identify Risks** — evaluated results against 6 standard API risk categories
- **Classify Severity** — rated each finding Low / Medium / High based on impact and exploitability
- **Document & Remediate** — compiled findings into this report with fix recommendations

Tools & Target

- API tested: DummyJSON — <https://dummyjson.com> (public, free, no signup)
- Testing tool: Postman (request builder, header inspection, response analysis)
- Reference material: OWASP API Security Top 10, API Security Checklist (study only)

Ethics & Scope

- Only read-only requests (GET) and safe authenticated POST requests (login) were performed
- No exploitation, flooding/DoS testing, or attacks were attempted
- All testing was limited to the public test API explicitly designated for this purpose

Finding 1 — Open/Unauthenticated Endpoints

GET /posts/1 — Publicly Readable With No Authentication

Category: Open / Unauthenticated Endpoints

LOW-MEDIUM RISK

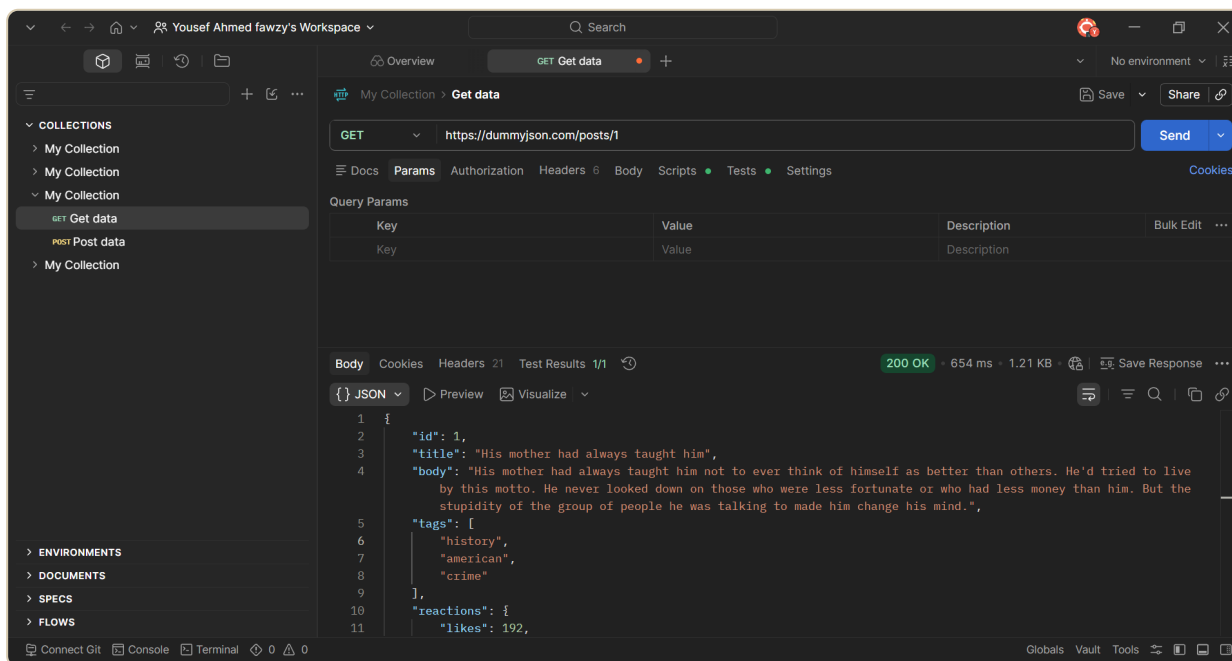


Fig 1.1 — GET /posts/1 returns full data with zero authentication (200 OK)

What Was Tested

A plain GET request was sent to /posts/1 with no API key, token, or login of any kind.

Result

Field	Observation
Authentication required	None
Response	200 OK — full post data returned (title, body, tags, reactions)

Business Impact

On its own, this endpoint's data (fake blog posts) is not sensitive, so the direct impact here is low. However, it demonstrates a broader pattern seen across this API: resources are world-readable by default with no access control layer — a pattern that becomes critical when applied to endpoints returning genuinely sensitive data (see Finding 2).

Risk Classification: Low-Medium. Not harmful in isolation given the non-sensitive content, but reflects an architecture with no default authentication requirement.

Finding 2 — Excessive Data Exposure

GET /users — Full PII, Credentials & Financial Data Leaked

Category: Excessive Data Exposure

CRITICAL / HIGH RISK

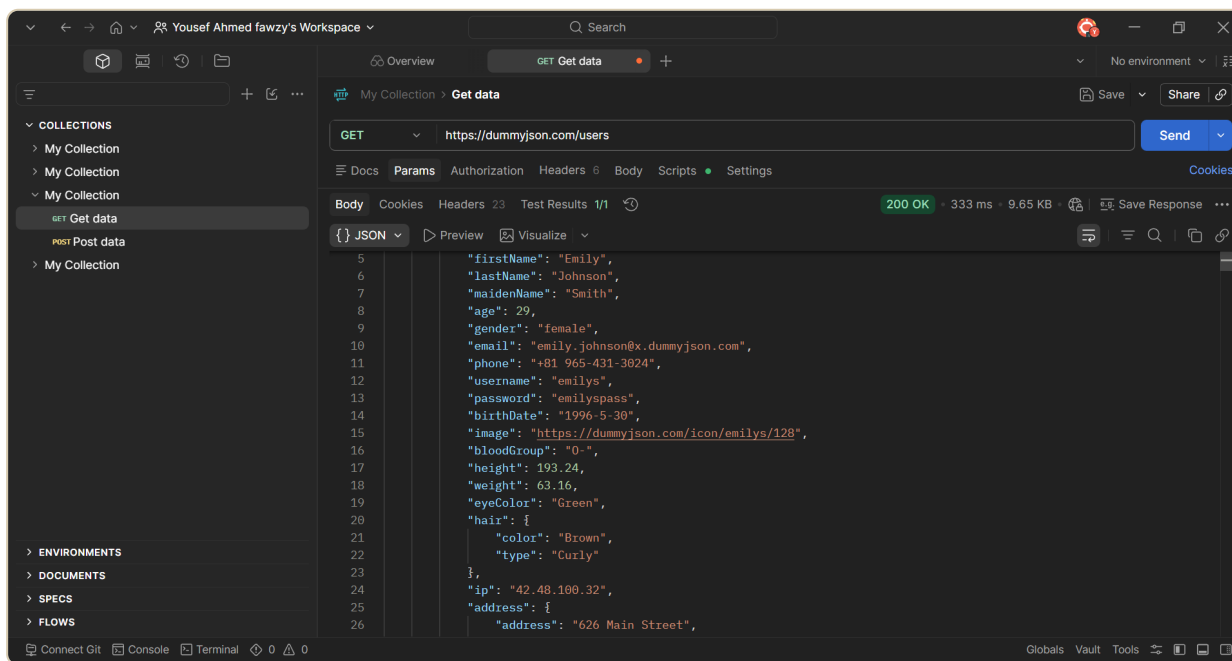


Fig 2.1 — GET /users returns deeply sensitive personal and financial fields with no authentication

What Was Tested

A plain GET request to /users, again with no authentication.

Sensitive Fields Exposed Per User

Field	Risk
password	Plaintext password — direct account takeover risk
ssn	Social Security Number — identity theft
bank.cardNumber, iban	Full payment card and banking details
crypto.wallet	Cryptocurrency wallet address
address + coordinates	Precise home location
macAddress, ip	Device fingerprinting data
role	Privilege level exposed (e.g. "admin")

Business Impact

If a production system exposed this endpoint as-is, any anonymous internet user could harvest every customer's login credentials, government ID number, and full banking details in a single request — with zero

technical barrier. This is a catastrophic, regulatory-grade data breach (GDPR/PCI-DSS violation territory) requiring no hacking skill whatsoever.

Risk Classification: Critical/High. This is the single most severe finding in this assessment — real PII, credentials, and financial data, fully exposed with no authentication.

Authentication Flow Testing

Before testing for weaknesses, the API's authentication mechanism itself was verified to understand how it is intended to work.

Step 1 — Successful Login

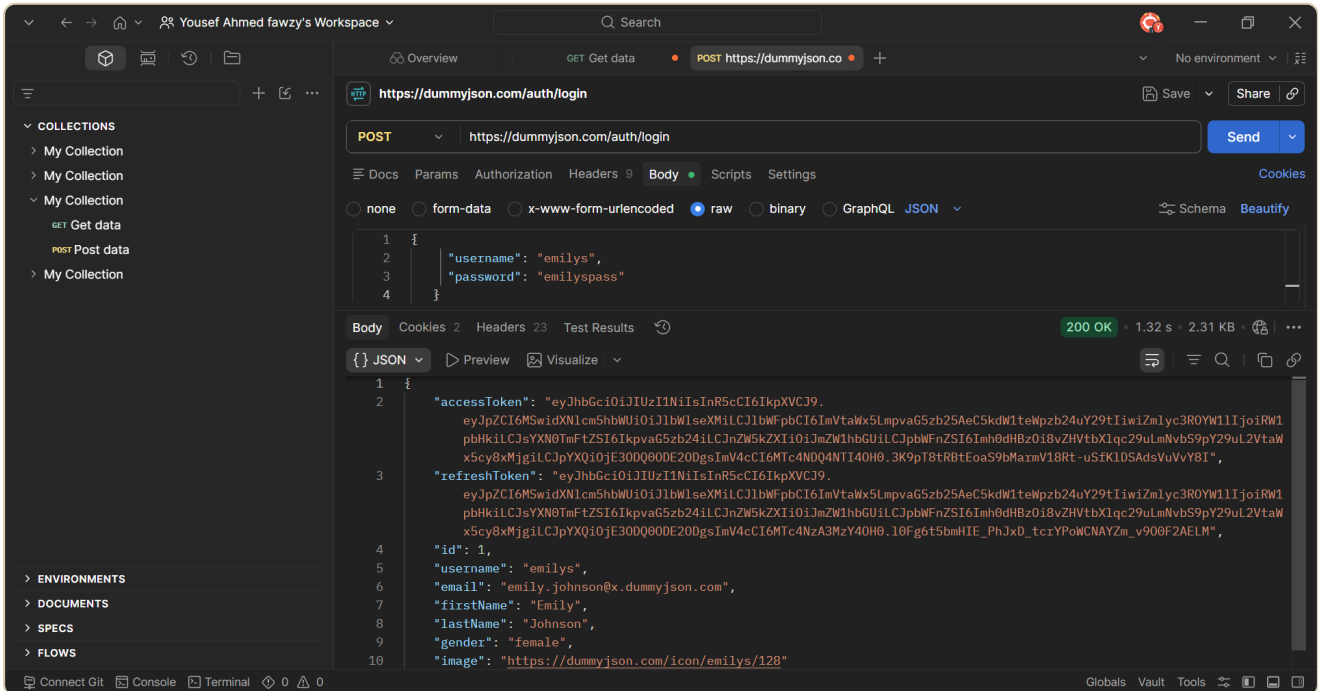


Fig 5.1 — POST /auth/login with valid credentials returns a signed JWT access token

A POST request with valid credentials returned **200 OK** along with a properly signed JWT `accessToken` and `refreshToken`.

Step 2 — Using the Token to Access Protected Data

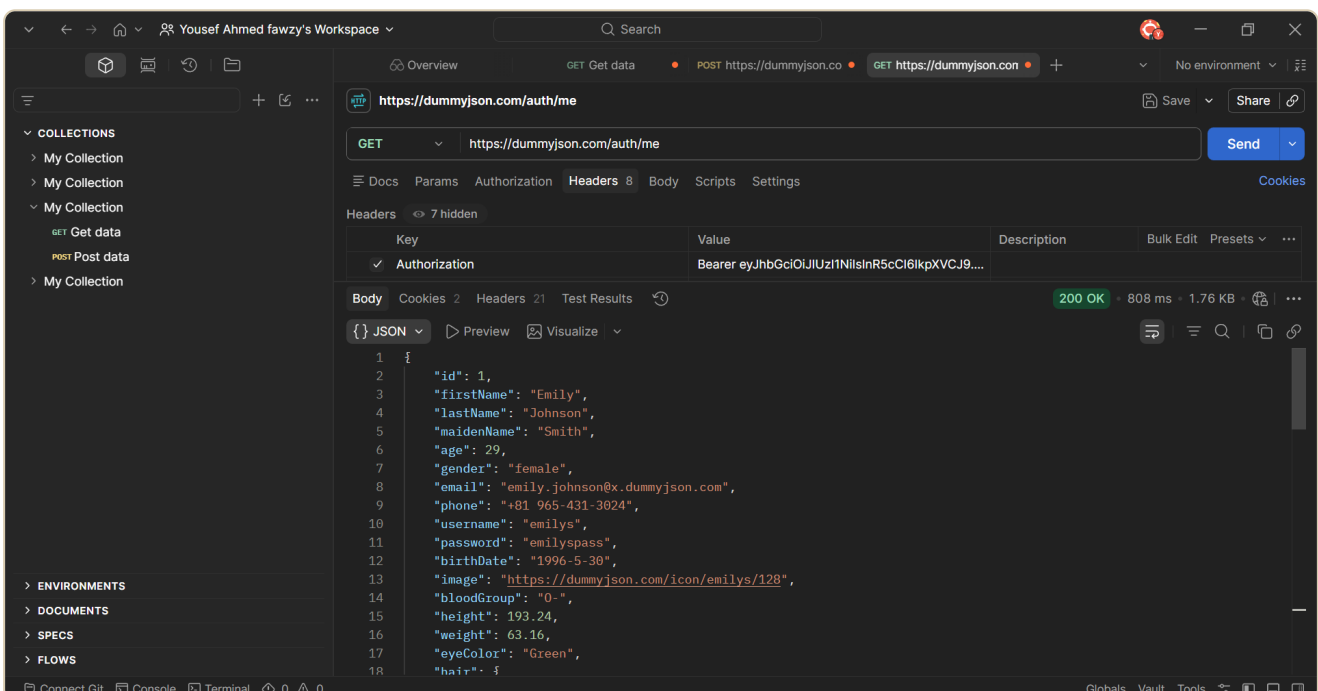


Fig 5.2 — GET /auth/me with a valid Bearer token correctly returns the authenticated user's profile

Passing the token via the Authorization: Bearer header successfully authenticated the request and returned the correct user's profile — confirming the token system functions as intended on the "happy path."

Step 3 — Access Attempt With No Token

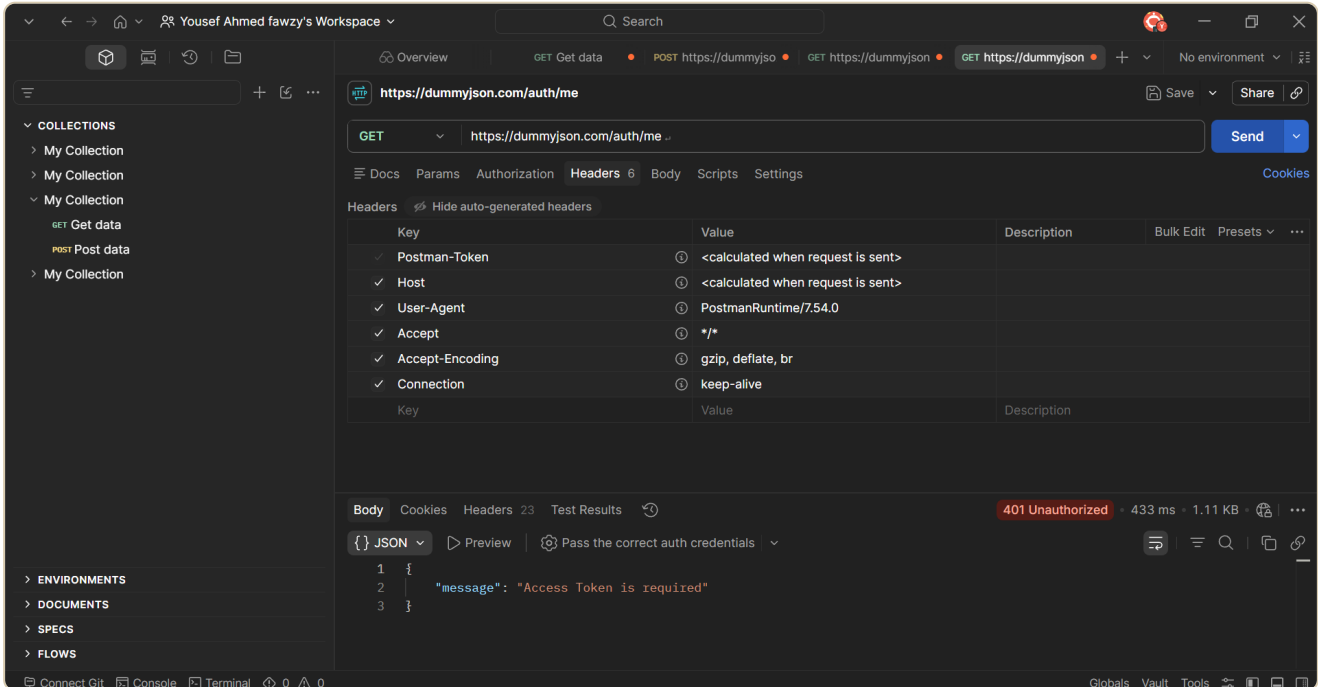


Fig 5.3 — Request with no Authorization header is correctly rejected (401)

Removing the Authorization header entirely resulted in 401 Unauthorized — "Access Token is required." Good security behavior.

Step 4 — Access Attempt With a Fake Token

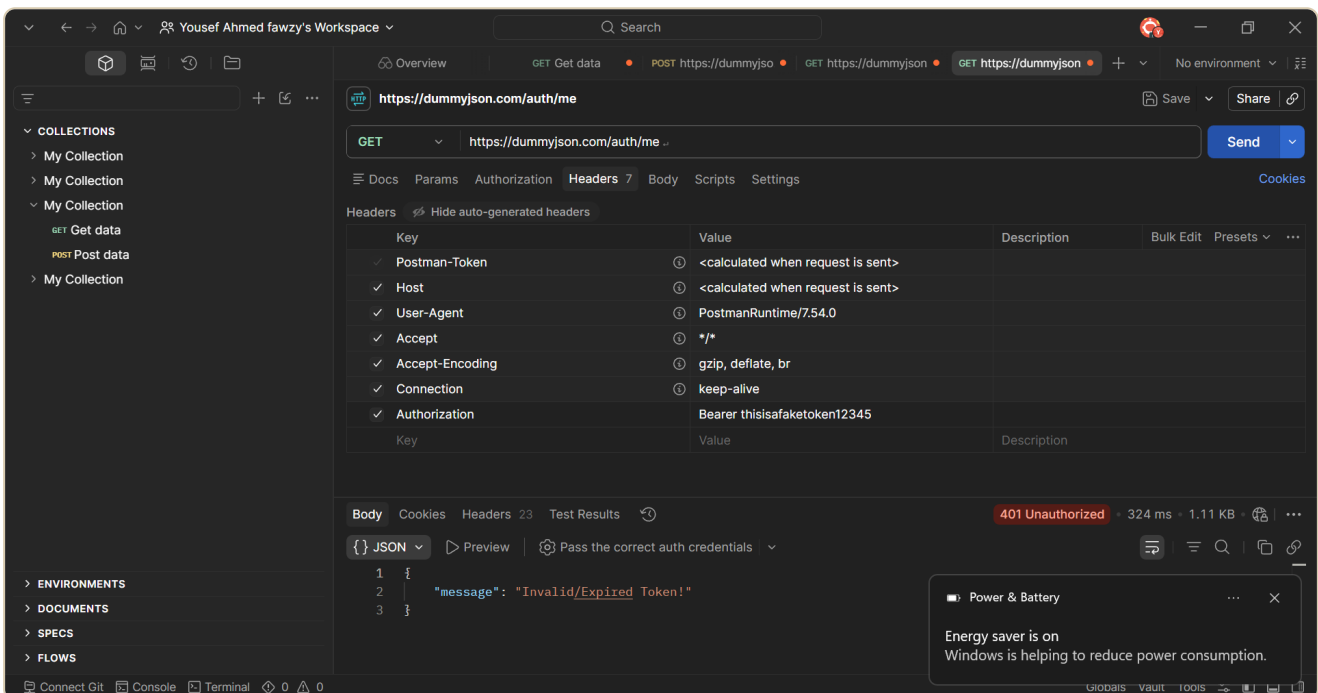


Fig 5.4 — A fabricated, invalid token is correctly rejected (401)

A made-up, non-existent token string was submitted. The API correctly identified it as invalid and returned **401 Unauthorized** — *"Invalid/Expired Token!"* — confirming genuine signature validation, not just a presence check.

Conclusion: The token-based authentication layer is correctly implemented — this is a positive security control, not a vulnerability.

Finding 3 — Broken Authorization (IDOR)

GET /users/2 — Any User's Full Profile Accessible by Guessing an ID

Category: Authorization Issues (Accessing Other Users' Data)

HIGH RISK

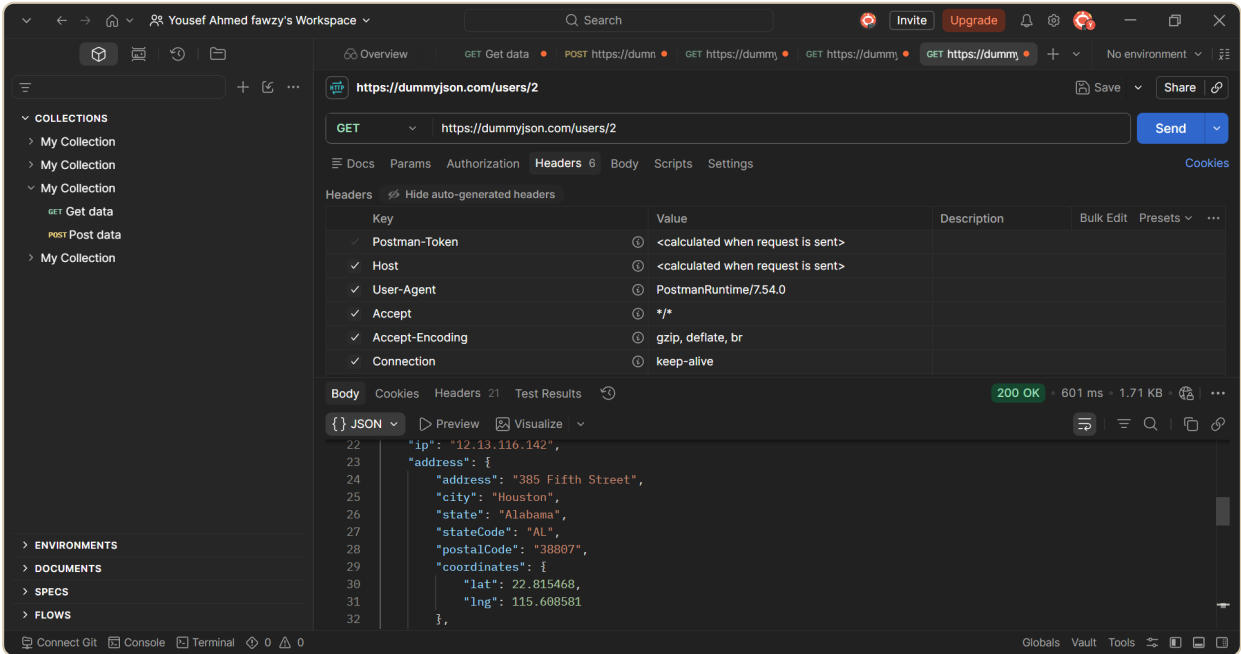


Fig 6.1 — Simply changing the ID in the URL exposes a completely different user's full sensitive record

What Was Tested

After observing user ID 1's data, the ID in the URL was manually changed to 2 — with no authentication and no ownership check of any kind.

Result

Field	Observation
Authorization check	None — no verification that the requester should see this record
Response	200 OK — full profile of a second, unrelated user returned (SSN, bank card, password, address)

Business Impact

This is a textbook **Insecure Direct Object Reference (IDOR)** vulnerability — one of the most common and dangerous flaws in real-world APIs. Combined with Finding 4 (no rate limiting), an attacker could trivially script a loop through every user ID and exfiltrate the entire customer database's identity and financial data.

Risk Classification: High. Directly enables mass data theft with a simple, automatable technique — no advanced skill required.

Finding 4 — Missing Rate Limiting

10 Rapid Sequential Requests — No Throttling Observed

Category: Missing Rate Limiting

MEDIUM RISK

What Was Tested

The GET `/users/2` endpoint was sent 10 times in rapid succession from the same client, to check whether the API detects and throttles automated/high-frequency request patterns.

Note: this test's live response could not be captured as a screenshot due to the rapid manual click sequence; the result is documented here based on direct observation during testing.

Result

Field	Observation
Requests sent	10, in rapid succession
Rate limit response (e.g. 429)	None observed
Response every time	200 OK, no delay, no CAPTCHA, no blocking

Business Impact

While not dangerous in isolation, the absence of rate limiting is a force-multiplier for Findings 2 and 3: it removes the last line of defense that would otherwise slow down or block an automated script attempting to scrape every user's sensitive data via the IDOR vulnerability.

Risk Classification: Medium. Amplifies the severity of other findings by enabling large-scale automated abuse.

Input Validation Testing

SQL-Injection-Style Login Attempt — Safely Rejected

Category: Input Validation

SECURE — NO ISSUE FOUND

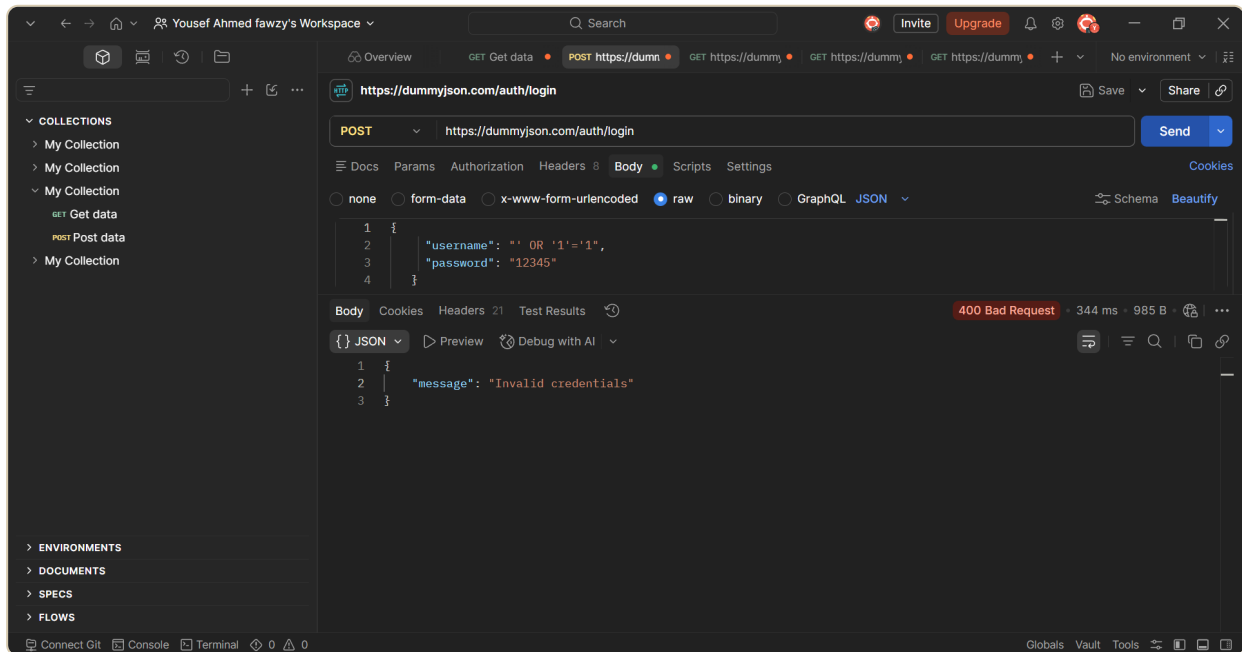


Fig 8.1 — Malicious-style input is correctly rejected with 400 Bad Request, not executed

What Was Tested

A login attempt was submitted with a classic SQL-injection-style payload (`' OR '1'='1`) in the username field, to check whether the API sanitizes and safely handles unexpected input.

Result

The API returned `400 Bad Request` — *"Invalid credentials"* — treating the payload as plain text rather than executable logic. No crash, no bypass, no unexpected behavior observed.

Conclusion: No input validation vulnerability was found on this endpoint. This is a positive security control.

Risk Summary & Remediation Plan

Finding	Category	Risk Level	Remediation
Open endpoints	No authentication	Low-Med	Require authentication by default on all data-returning endpoints
Excessive data exposure	Sensitive PII/financial leak	Critical	Field-level filtering; never return passwords/SSNs/bank data in API responses; hash passwords
Broken authorization (IDOR)	Access other users' data	High	Enforce server-side ownership checks; use non-sequential resource IDs (UUIDs)
Missing rate limiting	No throttling	Medium	Implement per-IP/per-token rate limits (e.g. 100 req/min) with temporary blocking
Token validation	Auth enforcement	Secure	No action needed — working as intended
Input validation	Injection handling	Secure	No action needed — working as intended

Key takeaway: This API's authentication mechanism itself is well-built — but that security is undermined by two endpoints (`/users` and `/users/{id}`) that bypass authentication entirely and expose the exact data the auth system is meant to protect. A security architecture is only as strong as its most exposed endpoint.

Conclusion & References

This assessment demonstrates a pattern common in real-world APIs: a properly engineered authentication system coexisting with data-exposing endpoints that never route through it. The two critical/high findings — excessive data exposure and broken authorization — together create a direct path for mass identity theft, further amplified by the absence of rate limiting. In contrast, the authentication and input validation layers were found to be correctly implemented, showing that strong security in one area does not guarantee it everywhere in an application.

For a production system, the highest-priority fix would be restricting the `/users` endpoints to require authentication and enforce per-user authorization, immediately followed by rate limiting to slow any automated abuse attempts.

References & Data Sources

- DummyJSON — Public test API used for this assessment (<https://dummyjson.com>)
- OWASP API Security Top 10 (Official Project) — referenced for risk category structure, study only
- API Security Checklist (shieldfy) — referenced for methodology guidance, study only

Note on ethics: All testing in this report was performed against a public API explicitly designated for learning and testing purposes. All requests were read-only or used safe test credentials; no exploitation, flooding, or attacks against private/production systems were performed at any point.